

- Document 05: Authentication & RBAC Middleware
 - CodePerfect Auditor — Zero-Trust Identity & Access Control Architecture
 - Overview
 - Layer 1: Identity & JWT — Supabase Auth
 - How Authentication Works
 - Linking auth.users to public.users
 - Layer 2: PostgreSQL RLS — Database-Kernel Isolation
 - The JWT Extraction Function
 - The RLS Policies
 - Why SECURITY DEFINER Matters
 - The Service Role Bypass
 - What RLS Means in Practice
 - Layer 3A: FastAPI — Configuration & Database Client Security
 - Centralized Settings with Pydantic Validation — config.py
 - Explanation
 - The Async Database Client — database.py
 - Paginated Queries with Authorization Headers
 - Layer 3B: Next.js — Frontend Route Guards & RBAC
 - The AuthProvider — Centralized Identity State
 - Navigation RBAC — Role-Filtered Menu Items
 - Explanation
 - Role Badge with Color Coding
 - CORS Configuration — API Boundary Security
 - Explanation
 - Complete Security Threat Model
 - API Route Structure

Document 05: Authentication & RBAC Middleware

CodePerfect Auditor — Zero-Trust Identity & Access Control Architecture

Overview

CodePerfect Auditor handles Protected Health Information (PHI) for multiple competing hospital organizations simultaneously. A single authentication misconfiguration could expose Hospital A's patient chart data to a rival hospital or insurance payer.

The security architecture enforces a **Zero-Trust, three-layer** identity model:

Layer	Technology	Responsibility
Layer 1	Supabase Auth (JWT)	User authentication and token issuance
Layer 2	PostgreSQL RLS	Database-kernel row isolation per organization
Layer 3	Next.js Route Guards + FastAPI Middleware	UI and API-level access control

No single layer is sufficient on its own. All three must pass for any data to be accessible.

Layer 1: Identity & JWT — Supabase Auth

How Authentication Works

Supabase Auth handles all credential management. The hospital coder or insurance adjudicator registers via `supabase.auth.signUp()` which:

1. Creates an `auth.users` row in Supabase's internal auth schema
2. Stores a bcrypt-hashed password (Supabase manages this — we never handle raw passwords)

3. Returns a signed **JWT** (JSON Web Token) containing the user's identity

The JWT payload looks like this (decoded):

```
{
  "sub": "a7f3b2c1-...", // Supabase auth.users UUID
  "email": "coder@citygeneral.com",
  "role": "authenticated",
  "app_metadata": {
    "organization_id": "e5d9c1a2-...", // Injected by our backend trigger
    "role": "coder" // Hospital RBAC role
  },
  "iat": 1743362400,
  "exp": 1743366000
}
```

Linking auth.users to public.users

When a user signs up, a PostgreSQL trigger fires that links the **auth.users** record to our **public.users** table via the **auth_id** foreign key:

```
-- From 002_core_tables.sql
-- This column bridges Supabase's internal auth system to our application users
table
ALTER TABLE public.users
  ADD COLUMN IF NOT EXISTS auth_id UUID
  UNIQUE REFERENCES auth.users(id) ON DELETE CASCADE;

-- Index for fast auth_id lookups – called on EVERY page load
CREATE INDEX IF NOT EXISTS idx_users_auth_id ON public.users(auth_id);
```

The **ON DELETE CASCADE** ensures that if a user is deleted from **auth.users** (e.g., HIPAA right-to-be-forgotten), their **public.users** record is automatically deleted by the database engine, preventing orphaned data.

Layer 2: PostgreSQL RLS — Database-Kernel Isolation

This is the most powerful security mechanism in the platform and the one that makes CodePerfect HIPAA-capable. Even if the FastAPI backend had a catastrophic

authorization bug, the database engine itself refuses to return data.

The JWT Extraction Function

```
-- From 006_audit_and_security.sql
-- This function reads organization_id directly from the user's JWT at the DATABASE
-- KERNEL level.
-- It does not trust anything the application layer sends – only the
-- cryptographically signed token.
CREATE OR REPLACE FUNCTION current_user_org_id() RETURNS UUID AS $$
BEGIN
    RETURN (
        current_setting('request.jwt.claims', true)::jsonb -- Read raw JWT claims
        -> 'app_metadata' -- Navigate to
    app_metadata object
        ->> 'organization_id' -- Extract
    organization_id string
    )::UUID;
EXCEPTION WHEN OTHERS THEN
    RETURN NULL; -- Return NULL (not an error) – policy USING clause will
-- evaluate to false
END;
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

The RLS Policies

```
-- Enable RLS on every table containing clinical data
ALTER TABLE clinical_cases ENABLE ROW LEVEL SECURITY;
ALTER TABLE coding_results ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_log      ENABLE ROW LEVEL SECURITY;

-- SELECT: a user can only read rows where organization_id matches their JWT
CREATE POLICY "org_isolation_clinical_cases_select"
ON clinical_cases FOR SELECT
USING (organization_id = current_user_org_id());

-- INSERT: a user can only write rows where organization_id matches their JWT
CREATE POLICY "org_isolation_clinical_cases_insert"
ON clinical_cases FOR INSERT
WITH CHECK (organization_id = current_user_org_id());

-- UPDATE: a user can only update their own org's rows
CREATE POLICY "org_isolation_clinical_cases_update"
ON clinical_cases FOR UPDATE
USING (organization_id = current_user_org_id());
```

Why SECURITY DEFINER Matters

```
-- The SECURITY DEFINER flag causes the function to execute with the privileges
-- of the FUNCTION OWNER (a trusted service role), not the calling user's
-- privileges.
-- This prevents a clever attacker from escalating privileges by manipulating
-- the session's role context before calling current_user_org_id().
$$ LANGUAGE plpgsql STABLE SECURITY DEFINER;
```

The Service Role Bypass

```
-- The FastAPI backend uses the supabase_service_key (service_role).
-- Service Role bypasses RLS – this is intentional.
-- The backend writes data ON BEHALF of users (e.g., pipeline writes clinical_cases
-- rows).
-- CRITICAL: The service_role key MUST NEVER be exposed to the browser/frontend.
-- From config.py:
#     supabase_service_key: str = "" # Only used server-side
```

What RLS Means in Practice

If a hacker obtains a valid JWT for **Org A** (City General Hospital) and attempts to query **coding_results** for **Org B** (Apollo Hospital):

```
-- Hacker's malicious query:
SELECT * FROM coding_results WHERE organization_id = 'apollo-org-uuid';

-- PostgreSQL internally rewrites this to:
SELECT * FROM coding_results
WHERE organization_id = 'apollo-org-uuid'
  AND organization_id = current_user_org_id(); -- ← RLS policy appended
-- automatically
-- current_user_org_id() returns 'city-general-uuid' (from hacker's JWT)
-- 'apollo-org-uuid' ≠ 'city-general-uuid' → PostgreSQL returns 0 rows
-- No error. No 403. No data. Silent, perfect isolation.
```

Layer 3A: FastAPI — Configuration & Database Client Security

Centralized Settings with Pydantic Validation — `config.py`

```
"""
config.py – Centralized configuration via Pydantic Settings.

All environment variables are validated here at startup.
Import `settings` everywhere instead of using os.getenv().
"""

from pydantic_settings import BaseSettings
from functools import lru_cache
from pydantic import field_validator

class Settings(BaseSettings):
    # — Supabase —————
    supabase_url: str = "" # Public endpoint – safe to log
    supabase_anon_key: str = "" # Public key – used in browser-facing
requests
    supabase_service_key: str = "" # SECRET – bypasses RLS, server-side only

    # — Groq LLM —————
    groq_api_key: str = "" # API key for clinical NLP model
    groq_model: str = "llama-3.3-70b-versatile"
    groq_timeout_seconds: int = 15 # Hard timeout to prevent hanging
pipeline
    groq_max_retries: int = 1

    # — Medical Standard Versions —————
    icd_version: str = "ICD-10-CM-2024"
    snomed_version: str = "SNOMED-CT-2024"

    # — WHO ICD API —————
    who_icd_client_id: str = "" # OAuth2 client credentials for WHO API
    who_icd_client_secret: str = ""

    @field_validator("supabase_url", "groq_api_key")
    @classmethod
    def validate_non_empty(cls, value: str) -> str:
        """Crash on startup if critical secrets are missing.
        This prevents the server from starting in a broken state where
        the AI pipeline would silently fail on every request."""
        if not value:
            raise ValueError("must not be empty.")
        return value

class Config:
```

```

env_file = ".env" # Loaded from .env in development
case_sensitive = False
extra = "ignore" # Ignore any extra env vars (security
hardening)

@lru_cache()
def get_settings() -> Settings:
    """Returns a CACHED Settings singleton.
    lru_cache ensures .env is parsed exactly ONCE at startup.
    This prevents timing attacks that could exploit re-parsing."""
    return Settings()

settings = get_settings() # Module-level alias – import this everywhere

```

Explanation

The `@lru_cache()` decorator is a security design choice, not just a performance optimization. If `get_settings()` was called without caching and the `.env` file was swapped on disk mid-request by an attacker with filesystem access, a naive implementation might pick up tampered credentials mid-request. The cached singleton is read once at startup and never re-read from disk.

The Async Database Client — `database.py`

```

# The database module maintains a SINGLE async HTTP client for all requests.
# Using a singleton prevents creating hundreds of TCP connections per request.
_client: httpx.AsyncClient | None = None

def _headers() -> dict:
    """
    Build the authorization headers for every Supabase REST API call.
    The service key is ALWAYS injected here – never passed as a parameter.
    This prevents any route handler from accidentally using the wrong key.
    """
    key = settings.supabase_service_key or settings.supabase_anon_key
    return {
        "apikey": settings.supabase_anon_key, # Required by all Supabase
requests
        "Authorization": f"Bearer {key}", # Service role for backend writes
        "Content-Type": "application/json",
    }

async def get_client() -> httpx.AsyncClient:
    """
    Returns the singleton async HTTP client. Creates it if needed.
    Uses Supabase's PostgREST REST endpoint – not a raw PostgreSQL socket.
    """

```

This means ALL queries go through PostgREST's authorization layer in addition to PostgreSQL's RLS policies (double validation).

"""

```
global _client
```

```
if _client is None or _client.is_closed:
```

```
    _client = httpx.AsyncClient(
```

```
        base_url=f"{settings.supabase_url}/rest/v1",
```

```
        headers=_headers(),
```

```
        timeout=10.0,    # Hard 10-second timeout prevents hanging DB queries
```

```
    )
```

```
    return _client
```

Paginated Queries with Authorization Headers

```
async def select_paginated(
```

```
    table: str,
```

```
    query: str = "*",
```

```
    filters: dict | None = None,
```

```
    order: str | None = None,
```

```
    page: int = 1,
```

```
    page_size: int = 20,
```

```
) -> tuple[list[dict], int]:
```

```
    """
```

```
    Paginated SELECT using PostgREST Range header.
```

```
    The Range header limits data exposure – no query ever returns unlimited rows,
    preventing accidental mass data leaks even if a filter is misconfigured.
```

```
    """
```

```
    offset = (page - 1) * page_size
```

```
    range_header = f"{offset}-{offset + page_size - 1}"
```

```
    response = await client.get(
```

```
        f"/{table}",
```

```
        params=params,
```

```
        headers={
```

```
            **_headers(),
```

```
            "Range": range_header,    # PostgREST range-based pagination
```

```
            "Range-Unit": "items",
```

```
            "Prefer": "count=exact",  # Returns total count in Content-Range
```

```
header
```

```
        },
```

```
    )
```

Layer 3B: Next.js — Frontend Route Guards & RBAC

The AuthProvider — Centralized Identity State

The `AuthProvider` React context is the single source of truth for user identity in the Next.js frontend. Every protected page reads from it.

The hospital `layout.tsx` enforces both authentication and role separation:

```
// frontend/src/app/hospital/layout.tsx
'use client';

import { useEffect } from 'react';
import { useRouter } from 'next/navigation';
import { useAuth } from '@components/AuthProvider';

export default function HospitalLayout({ children }: { children: React.ReactNode }) {
  const { orgUser, org, loading, signOut } = useAuth();
  const router = useRouter();

  useEffect(() => {
    // Guard 1: Unauthenticated users are immediately redirected to login.
    // This runs on every page load, not just the first.
    if (!loading && !orgUser) router.push('/auth/login');

    // Guard 2: Payer users attempting to access the hospital portal
    // are redirected to their own portal – even if they have a valid JWT.
    // A payer adjudicator has NO business inside a hospital's coding
    workspace.
    if (!loading && orgUser?.role === 'payer') router.push('/payer/inbox');
  }, [loading, orgUser, router]);

  if (loading) return <LoadingSpinner />;
  if (!orgUser) return null; // Prevents any flash of protected content

  return (
    <div className="min-h-screen flex">
      <aside>
        {/* Navigation items filtered by RBAC role */}
      </aside>
      <main>{children}</main>
    </div>
  );
}
```

Navigation RBAC — Role-Filtered Menu Items

```
// Hospital navigation items – each item declares which roles can see it
const navItems = [
```

```

    {
      href:      '/hospital/coder/analyze',
      name:      'New Analysis',
      allowedRoles: ['coder', 'admin'], // Only coders and admins can submit new
cases
    },
    {
      href:      '/hospital/rcm/inbox',
      name:      'Claims Inbox',
      allowedRoles: ['rcm', 'admin'], // Only RCM staff and admins see the
claims queue
    },
    {
      href:      '/hospital/rcm/analytics',
      name:      'Analytics',
      allowedRoles: ['rcm', 'admin'], // Financial analytics hidden from
coders
    },
    {
      href:      '/hospital/admin/branches',
      name:      'Branches',
      allowedRoles: ['admin'], // User management: admin-only
    },
    {
      href:      '/hospital/admin/users',
      name:      'Users',
      allowedRoles: ['admin'], // Branch management: admin-only
    },
  ],

  // In the render function – server-side filtered navigation
  {navItems.map(item => {
    // If the user's role is NOT in allowedRoles, return null.
    // The menu item is completely removed from the DOM – not hidden with CSS.
    // CSS-hidden items can still be found via dev tools; null-returned items
cannot.
    if (!item.allowedRoles.includes(orgUser.role)) return null;
    return <NavLink key={item.href} {...item} />;
  })}

```

Explanation

Returning `null` (completely removing from DOM) rather than `display: none` is a deliberate security engineering decision. A CSS-hidden navigation item can be re-enabled by any user opening browser DevTools and deleting the `hidden` attribute. A `null` React return produces no DOM node whatsoever — preventing UI-level privilege escalation entirely.

Role Badge with Color Coding

```
// Visual indicator of the user's current role – color-coded for instant
recognition
<div className={`text-[10px] font-semibold uppercase tracking-wider ${
  orgUser.role === 'admin'    ? 'text-amber-400'    :
  orgUser.role === 'auditor'  ? 'text-blue-400'     :
  orgUser.role === 'rcm'      ? 'text-orange-400'   :
  orgUser.role === 'payer'    ? 'text-emerald-400'  :
  'text-emerald-400' // coder (default)
}`}>
  {orgUser.role}  {/* Displayed in the sidebar footer */}
</div>
```

CORS Configuration — API Boundary Security

```
# backend/main.py
# CORS (Cross-Origin Resource Sharing) restricts which browser origins
# are allowed to make API calls to the FastAPI backend.
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",    # Next.js development
        "http://localhost:3001",    # Alternate dev port
        "http://127.0.0.1:3000",
        "http://127.0.0.1:3001",
        # Production: replace with "https://codeperfect.hospital.com"
    ],
    allow_credentials=True,          # Allows cookies (JWT refresh tokens)
    allow_methods=["*"],             # All HTTP verbs allowed from trusted origins
    allow_headers=["*"],             # All headers allowed (Authorization, Content-Type)
)
```

Explanation

Restricting `allow_origins` to the specific frontend domain prevents **Cross-Site Request Forgery (CSRF)** attacks. If `allow_origins=["*"]` was used (a common beginner mistake), any malicious website could make authenticated API calls on behalf of a logged-in hospital coder — potentially reading or modifying patient coding records.

Complete Security Threat Model

Threat	Layer 1 Mitigation	Layer 2 Mitigation	Layer 3 Mitigation
Unauthenticated access	Supabase JWT required	RLS returns 0 rows	Router redirects to login
Cross-tenant data access	JWT has org_id claim	RLS enforces org_id at kernel	Not possible if JWT correct
Payer accessing hospital data	JWT has role=payer	Separate claims/coding_results queries	Route guard → /payer/inbox
Coder accessing admin pages	JWT has role=coder	No DB-level difference	navItems filtered null
API abuse (no browser)	Token required in header	RLS on every query	CORS blocks unknown origins
Compromised service_key	Not applicable	Service role bypass limited to backend	Key never sent to browser
Session hijacking	JWT short expiry (1 hour)	JWT org_id still verified	Supabase refresh token rotation

API Route Structure

```
FastAPI Routes (prefix: /api/v1)
├─ /health                → GET  - No auth required (monitoring)
├─ /icd/*                 → GET  - Read ICD lookup data (anon key)
├─ /code/run              → POST - Trigger AI pipeline (anon key + org_id from body)
├─ /code/run-pdf          → POST - Upload PDF, trigger AI pipeline
├─ /cases/*               → GET  - Case history (anon key + RLS enforces org isolation)
├─ /claims/submit         → POST - Submit claim to payer (service_key for DB write)
└─ /claims/{id}/adjudicate → POST - Payer adjudication (service_key)
```

— /claims/{id}/edi-export	→ GET	– Download EDI 837 (service_key)
— /analytics/*	→ GET	– Financial analytics (service_key + org filter)
— /payers/*	→ GET	– Payer configuration (anon key)